ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ
Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»
(СПб ГБПОУ «Петровский колледж»)
Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК.02.01 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ № 3»

Выполнил: Бокатый Григорий Владимирович
Студент группы: 54-15
Специальность: 09.02.08 Интеллектуальные интегрированные системы
Руководитель: Борисов Егор Николаевич
Оценка: _____

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ
СПб ГБПОУ «Петровский колледж»
Отделение информационных технологий

УТВЕРЖДАЮ
Заведующий отделением
информационных технологий
А.Ю. Бычков _____
«____» _____ 2026 г.

ЗАДАНИЕ

для выполнения курсового проекта
Специальность: 09.02.08 «Интеллектуальные интегрированные системы»
Группа: 54-15
Ф.И.О. студента: Бокатый Григорий Владимирович

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант № 3»

Разработка принципиальной схемы, виртуальной модели Arduino Uno, прошивки, базы данных SQLite и обработки данных в Node-RED; ведение проекта в Git.

Входные данные варианта 3

Параметр	Значение
Сценарий	Серверная
Датчик 1	NTC — температура, 0…60 °C
Датчик 2	MQ2 — газ / дым, условная шкала 0…100 %
Датчик 3	PIR — движение, 0 или 1
Исполнительное устройство	Реле — вентилятор
Светодиод-индикатор	Зелёный, резистор 150 Ом
Период опроса T	3 с
Порог события	Температура s1 > 27 °C

Содержание курсового проекта

Введение
1. Постановка задачи
2. Структура и алгоритм работы системы
3. Разработка аппаратной части
4. Разработка прошивки
5. Проектирование базы данных
6. Загрузка и контроль данных в Node-RED
7. Тестирование и результаты
8. Расчёт объёма данных
9. Управление версиями Git
10. Предложения по модернизации

Выводы; список сокращений; список литературы

Приложения: структурная схема, модель Wokwi, листинги, журналы и история Git

Руководитель проекта _____ (Борисов Е.Н.)

Студент _____ (Бокатый Г.В.)

Содержание

Введение 5
1. Постановка задачи 7
2. Структура и алгоритм работы системы 9
  2.1. Структурная схема 9
  2.2. Алгоритм функционирования 10
  2.3. Протокол обмена данными 11
3. Разработка аппаратной части 12
  3.1. Выбор компонентов и подключение 12
  3.2. Расчёт резистора и оценка питания 13
  3.3. Принципиальная схема в EasyEDA 15
  3.4. Модель в Wokwi 15
  3.5. Расчёт шага АЦП 16
4. Разработка прошивки 17
5. Проектирование базы данных 18
6. Загрузка и контроль данных в Node-RED 20
7. Тестирование и результаты 22
8. Расчёт объёма данных 26
9. Управление версиями Git 27
10. Предложения по модернизации 28
Выводы 29
Список используемых сокращений 30
Список используемой литературы 31
Приложение А. Структурная схема 32
Приложение Б. Схема электрическая принципиальная 33
Приложение В. Перечень элементов 34
Приложение Г. Модель в Wokwi 35
Приложение Д. Листинг программы Arduino 36
Приложение Е. Листинг функции Node-RED 37
Приложение Ж. Скрипты базы данных 39
Приложение И. Журналы и загрузочный скрипт 40
Приложение К. История Git 42

Введение

Интеллектуальные интегрированные системы объединяют датчики, микроконтроллеры, исполнительные устройства и программные средства обработки информации. Помимо измерения параметров важны сохранение результатов, регистрация событий и контроль корректности показаний.

В серверной необходимо контролировать температуру оборудования и воздуха, наличие дыма и движение в помещении. Повышение температуры может указывать на нарушение охлаждения, поэтому в учебной модели предусмотрен канал управления вентилятором и световая индикация.

В проекте применяется Arduino Uno, а работу устройства проверяют в виртуальной среде Wokwi. Для сбора данных используются текстовые сообщения последовательного порта, Node-RED и база SQLite. Файлы проекта сохраняются в Git [1–6].

Объект проекта — виртуальная система сбора и учёта данных для серверной. Предмет проекта — подключение датчиков, программа Arduino, протокол обмена и механизм загрузки данных в базу.

Цель проекта — разработать и описать систему варианта № 3, выполняющую опрос трёх датчиков с периодом 3 с, регистрацию превышения температурного порога и проверку данных при загрузке.

Для достижения цели необходимо решить следующие задачи:

– определить состав датчиков, допустимые диапазоны и порог срабатывания;

– разработать структурную схему, алгоритм и модель подключения к Arduino Uno;

– выполнить расчёт резистора светодиода и оценку разрешения АЦП;

– разработать прошивку и текстовый протокол DATA/EVENT;

– спроектировать таблицы SQLite и контрольные запросы;

– настроить обработку журнала в Node-RED и проверить регистрацию неисправности;

– оценить объём данных и отразить фактическую историю версий.

Работа описывает виртуальную модель серверной: Arduino считывает показания датчиков, управляет реле и индикатором, а программная часть обрабатывает и сохраняет данные. Для MQ2 используется относительная шкала 0–100 %.

1. Постановка задачи

По варианту № 3 разрабатывается система контроля серверной. Датчик NTC используется для оценки температуры, MQ2 — для наблюдения за условным уровнем газа или дыма, PIR — для регистрации движения. Реле представляет канал управления вентилятором, а зелёный светодиод показывает состояние температурного порога.

Таблица 1 — Входные данные варианта № 3

Параметр	Значение
Сценарий	Серверная
Датчик 1	NTC — температура, 0…60 °C
Датчик 2	MQ2 — газ / дым, условная шкала 0…100 %
Датчик 3	PIR — движение, 0 или 1
Исполнительное устройство	Реле — вентилятор
Светодиод-индикатор	Зелёный, резистор 150 Ом
Период опроса T	3 с
Порог события	Температура s1 > 27 °C

В каждом цикле программа должна формировать строку DATA с номером n и тремя значениями. При переходе температуры выше 27 °C выводы D7 и D8 устанавливаются в HIGH и передаётся EVENT THRESHOLD. При возвращении температуры к 27 °C или ниже выводы переводятся в LOW и передаётся EVENT NORMAL.

События формируются при изменении состояния, а не повторяются в каждом цикле. Данные Serial Monitor пользователь вручную копирует в data/log.txt; далее поток Node-RED подготавливает SQL для загрузки в db/variant.db.

Для контроля допустимости используются диапазоны s1: 0…60 °C, s2: 0…100 %, s3: только 0 или 1. Числовое значение вне диапазона сохраняется в readings, а в events создаётся FAULT с названием датчика. Некорректное или отсутствующее значение также вызывает FAULT, но в readings не записывается.

Температурный порог и диапазон достоверности не совпадают: значение 42,46 °C допустимо как показание, но превышает порог управления. Значение 999 °C недопустимо и создаёт FAULT при обработке журнала. Проверка диапазонов выполняется на стороне Node-RED, а не в прошивке Arduino.

Для проверки базы предусмотрены три запроса: десять последних записей показаний, минимум/среднее/максимум по каждому датчику и перечень событий.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система включает три датчика, микроконтроллер Arduino Uno, релейный выход, светодиод и программную цепочку сохранения данных. NTC подключён к A0, MQ2 — к A1, PIR — к D2. Управляющие сигналы реле и индикатора поступают соответственно на D7 и D8.

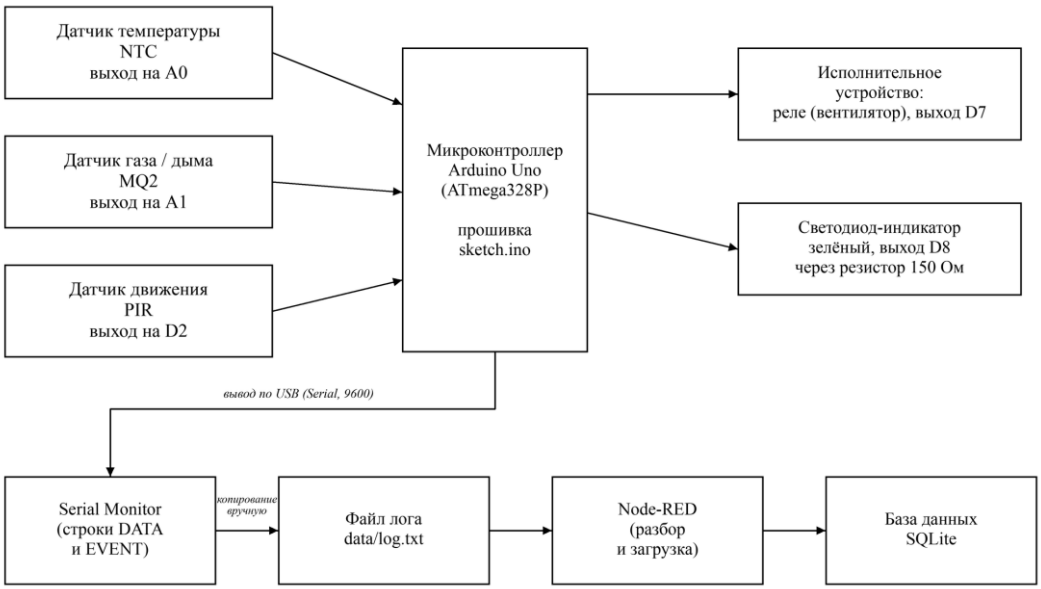


Рисунок 1 — Структурная схема системы

Строки из Serial Monitor копируются пользователем в файл журнала. Узлы Node-RED считывают этот файл и формируют скрипт для SQLite. Копирование является ручным этапом; непосредственное подключение Node-RED к Serial в данном проекте не предусмотрено.

2.2. Алгоритм функционирования

После запуска программа настраивает последовательный порт и выводы. Переменные n и lastMs равны нулю, флаг thresholdActive сброшен. Проверка времени осуществляется через millis(): если прошло менее 3000 мс, текущий проход loop() завершается без опроса.

После наступления очередного интервала счётчик увеличивается, читаются NTC, MQ2 и PIR, затем формируется DATA. Если температура впервые стала выше 27 °C, выставляются HIGH на D7 и D8 и передаётся THRESHOLD. При возвращении к норме устанавливаются LOW и передаётся NORMAL. Алгоритм представлен на рисунке 2.

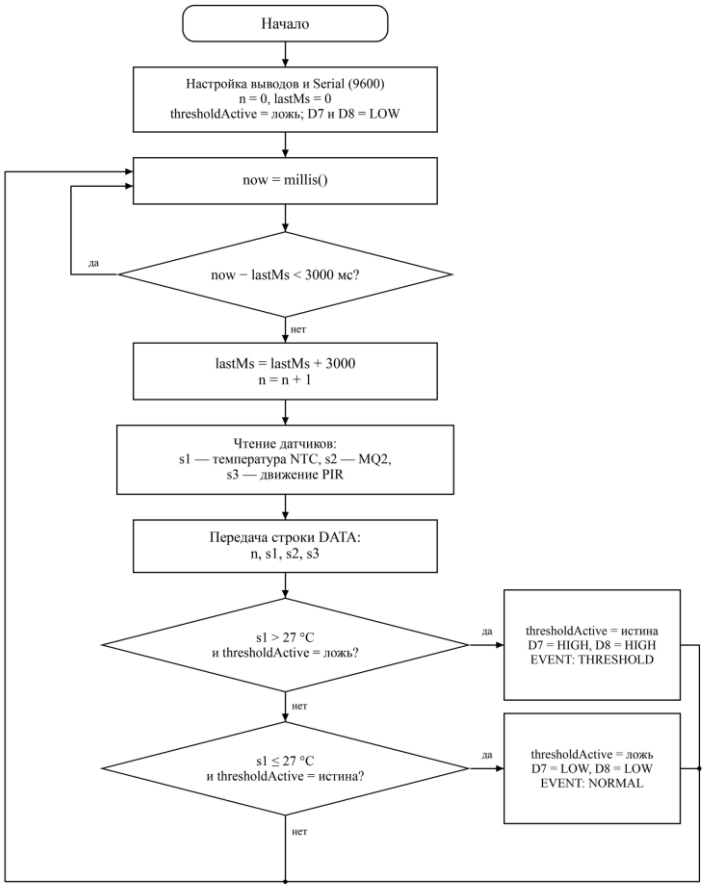


Рисунок 2 — Алгоритм программы Arduino

2.3. Протокол обмена данными

Скорость последовательного порта — 9600 бод. Используются текстовые строки с полями в формате «ключ=значение». Десятичным разделителем является точка. Номер n связывает показания и события одного цикла.

Таблица 2 — Формат строк Serial

Тип	Пример
DATA	DATA n=6 s1=42.46 s2=88.66 s3=0
EVENT	EVENT n=6 type=THRESHOLD
EVENT	EVENT n=11 type=NORMAL

Поле s1 содержит температуру в °C, s2 — относительное значение MQ2 в процентах шкалы, s3 — состояние PIR. Первые два значения выводятся с двумя знаками после точки. Время в базе рассчитывается как $t_s = n \cdot 3$ и является условным временем от начала работы программы, а не календарной отметкой.

Событие FAULT не поступает от Arduino: его создаёт обработчик журнала, если значение отсутствует, не является конечным числом либо выходит за установленные границы. После перезапуска Arduino нумерация начинается заново.

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

В модели используются плата Arduino Uno на ATmega328P, модуль терморезистора NTC, датчик MQ2, датчик движения PIR, модуль реле и зелёный светодиод с токоограничивающим резистором. Питание сигнальных модулей организовано от 5 V и GND платы.

Таблица 3 — Подключение к Arduino Uno

Сигнал	Компонент	Вывод	Режим
Температура s1	NTC, OUT	A0	Аналоговый вход
Газ / дым s2	MQ2, AOUT	A1	Аналоговый вход
Движение s3	PIR, OUT	D2	Цифровой вход
Вентилятор	Реле, IN	D7	Управляющий выход
Индикация	Зелёный LED, 150 Ом	D8	Цифровой выход
Питание	Модули датчиков и реле	5 V, GND	Общие шины

MQ2 в прошивке преобразуется по формуле s2 = 100 · analogRead(A1) / 1023. Это условная линейная шкала кода АЦП, а не калиброванная концентрация. Температура вычисляется по нелинейной BETA-модели NTC с коэффициентом 3950 K.

3.2. Расчёт резистора и оценка питания

Для зелёного светодиода принимаются напряжение питания 5 В, прямое напряжение 2,0 В и ток 20 мА. Сопротивление и мощность резистора:

$$R1 = (5 - 2{,}0) / 0{,}020 = 150 \text{ Ом};$$

$$P = 0{,}020^2 \cdot 150 = 0{,}060 \text{ Вт}.$$

Выбран R1 = 150 Ом мощностью 0,25 Вт. Включённая цепь LED1–R1 при принятых значениях потребляет ток 20 мА и мощность от шины 5 В, равную 0,100 Вт.

Расчёт питания выполняется при включённых реле и индикаторе. Как в приведённом образце, для Arduino Uno принято 50 мА. Для MQ2 при мощности нагревателя до 900 мВт расчётный ток нагревателя равен 0,9 / 5 = 0,18 А; с резервом 10 мА на электронику принято 190 мА.

Для NTC принят делитель с постоянным резистором 10 кОм и терморезистором 10 кОм при 25 °C: I = 5 / 20 000 = 0,25 мА. Предельный ток такого делителя составляет 0,5 мА; с резервом 1 мА на электронику для оценки принято 1,5 мА. Максимальная мощность на NTC в этом делителе равна $5^2 / (4 \cdot 10\,000) = 0{,}625$ мВт.

Для одноканального релейного модуля K1, как в образце, принято 80 мА: 72 мА на катушку 5 В и 8 мА на цепь управления и индикацию. Катушка питается от шины 5 В; D7 передаёт управляющий сигнал.

По документации PIR напряжение питания составляет 3–5 В, предельный выходной ток при Vdd = 5 В — 10 мА [10]. Для расчёта потребления модуля PIR принят ток 2 мА.

Таблица 4 — Расчёт потребления от шины 5 В

Потребитель	Ток, мА	Мощность, Вт	Основание
Arduino Uno	50	0,250	Принято по образцу
MQ2, модуль	190	0,950	180 мА нагреватель + 10 мА резерв
NTC, измерительная цепь	1,5	0,0075	0,5 мА делитель + 1 мА резерв
PIR, модуль	2	0,010	Принято для расчёта
Реле K1, включено	80	0,400	72 мА катушка + 8 мА управление
LED1 с R1	20	0,100	Расчёт при Uпр = 2,0 В
Итого	343,5	1,7175	При принятых токах

Расчётный суммарный ток IΣ = 343,5 мА; мощность PΣ = 5 · 0,3435 = 1,7175 Вт. С запасом 30 % требуется около 446,6 мА, что меньше 500 мА. Для отдельного питания выбран стабилизированный источник 5 В / 1 А.

При непрерывном максимальном расчётном режиме за сутки E = 1,7175 · 24 = 41,22 Вт·ч. Двигатель вентилятора учитывается отдельно по силовой цепи.

3.3. Принципиальная схема в EasyEDA

Принципиальная схема разработана в EasyEDA и представлена в файле docs/schematic.pdf. На ней обозначены A1 — Arduino Uno, B1 — датчик газа MQ2, B2 — датчик температуры NTC, U1 — датчик движения PIR, K1 — модуль реле, LED1 — зелёный светодиод, R1 — резистор 150 Ом. Состав компонентов также приведён в docs/bom.csv.

При разработке схемы размещаются компоненты, подключаются шины +5 V и GND, после чего соединяются сигнальные выводы датчиков и исполнительных устройств. Светодиод LED1 включён последовательно с токоограничивающим резистором R1. На схеме контакты реле оставлены для последующего подключения нагрузки.

3.4. Модель в Wokwi

Виртуальная модель включает Arduino Uno, датчики NTC, MQ2 и PIR, релейный модуль, зелёный светодиод и резистор 150 Ом. Соединения определены в sim/diagram.json. Температура, уровень газа и движение задаются в элементах симулятора. Снимок модели приведён в приложении Г, а работа симуляции — в разделе 7.

В модели управляющий вход реле подключён к D7, светодиод через резистор — к D8. Реле представляет канал управления вентилятором. Текст из Serial Monitor сохраняется пользователем в data/log.txt для последующей обработки.

3.5. Расчёт шага АЦП

АЦП Arduino Uno имеет 10 разрядов и 1024 уровня, а диапазон кодов составляет 0…1023. При опорном напряжении 5 В номинальный шаг квантования напряжения равен:

$$\Delta U = 5 / 1024 \approx 0{,}004883 \text{ В} = 4{,}88 \text{ мВ}.$$

Для относительной шкалы MQ2 соседние коды изменяют значение на 100 / 1023 ≈ 0,0978 процентного пункта. Деление на 1023 позволяет отобразить максимальный код как 100 %.

Температура NTC в программе рассчитывается нелинейно; поэтому деление 60 °C на 1023 не является фактическим постоянным шагом температуры. Вблизи 27 °C соседние коды модели дают приблизительно 0,09 °C, однако шаг зависит от температуры и параметров делителя.

4. Разработка прошивки

Программа firmware/sketch.ino написана на C++ с использованием Arduino API. Константы задают период T_MS = 3000 мс, коэффициент BETA = 3950 и порог TEMP_THRESHOLD = 27 °C. Исходный текст приведён в приложении Д.

Таблица 5 — Основные функции и переменные

Элемент	Назначение
setup()	Настройка Serial 9600, D2, D7 и D8; сброс выходов
loop()	Контроль времени, чтение датчиков, выдача DATA/EVENT
readTemperature()	Преобразование кода NTC по BETA-модели
lastMs, T_MS	Определение момента следующего опроса
thresholdActive	Хранение предыдущего состояния порога
n	Номер цикла измерения

Функция readTemperature() ограничивает код АЦП диапазоном 1…1022 перед вычислением, что предотвращает крайние значения в выражении. На A1 выполняется линейное преобразование в относительную шкалу, а D2 считывается функцией digitalRead().

В loop() используется неблокирующая проверка millis(). При наступлении периода lastMs увеличивается на 3000; первый опрос происходит примерно через 3 секунды после запуска. Значения s2 и s3 регистрируются, но не участвуют в условии переключения выхода D7.

Флаг thresholdActive предотвращает повтор событий во время длительного превышения. Значение ровно 27 °C относится к условию возврата.

5. Проектирование базы данных

5.1. Модель данных

Для локального хранения выбрана SQLite [5]. В базе db/variant.db используются таблицы readings и events. Каждое показание отдельного датчика занимает отдельную строку. Таблицы логически сопоставляются по номеру n; внешний ключ в схеме не задан.

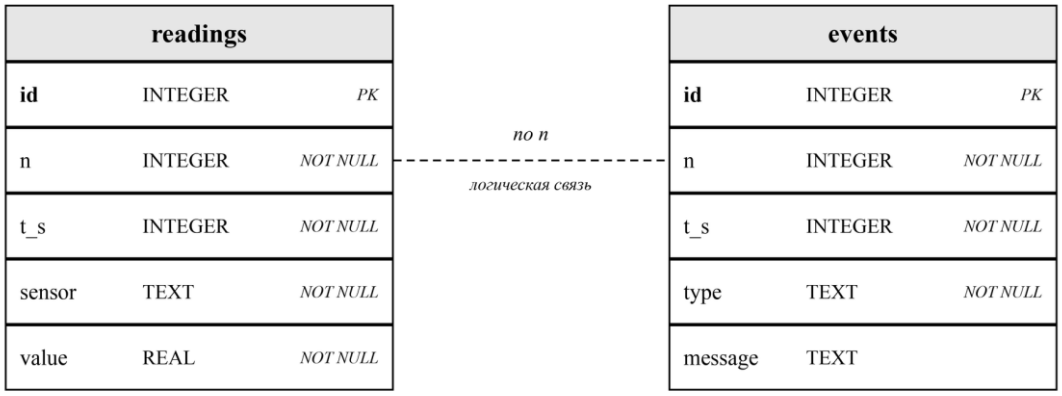


Рисунок 3 — Модель данных SQLite

5.2. Описание таблиц

Таблица 6 — Таблица readings

Поле	Тип	Назначение
id	INTEGER, PK	Идентификатор записи
n	INTEGER NOT NULL	Номер измерения
t_s	INTEGER NOT NULL	Условное время, с
sensor	TEXT NOT NULL	Идентификатор s1, s2 или s3
value	REAL NOT NULL	Числовое показание

Таблица 7 — Таблица events

Поле	Тип	Назначение
id	INTEGER, PK	Идентификатор события
n	INTEGER NOT NULL	Номер цикла события
t_s	INTEGER NOT NULL	Условное время, с
type	TEXT NOT NULL	THRESHOLD, NORMAL или FAULT
message	TEXT	Пояснение к событию

Скрипт schema.sql использует CREATE TABLE IF NOT EXISTS и допускает повторное создание структуры. Значения, выходящие за пределы диапазона, не запрещены ограничением CHECK: числовое показание сохраняется и сопровождается событием FAULT. Поэтому итоговые средние значения следует анализировать с учётом аварийных записей.

5.3. Контрольные запросы

Первый запрос выводит 10 последних строк readings по убыванию id. Поскольку один цикл даёт до трёх отдельных строк, это не обязательно десять циклов. Второй запрос группирует показания по sensor и вычисляет минимум, среднее и максимум. Третий выводит события по порядку добавления. SQL-скрипты приведены в приложении Ж.

6. Загрузка и контроль данных в Node-RED

6.1. Состав потока

Для обработки файла используется поток Node-RED: отслеживание или ручной запуск, чтение журнала, функция разбора, запись SQL и запуск sqlite3. Результат выполнения и ошибки выводятся в debug-узлы. Снимок предоставленной сборки приведён на рисунке 4.

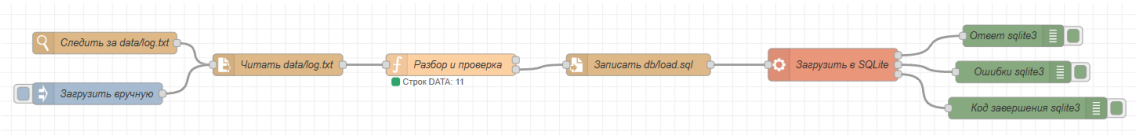


Рисунок 4 — Поток Node-RED

Таблица 8 — Назначение узлов

Узел	Назначение
watch / inject	Запуск при изменении файла или вручную
file in	Чтение data/log.txt целиком
function	Разбор DATA/EVENT, проверка, формирование SQL
file	Запись db/load.sql
exec	Запуск sqlite3 со схемой и загрузочным скриптом
debug	Просмотр вывода, ошибок и кода завершения

Полный поток с узлами и соединениями представлен в файле nodered/flow.json. Узел функции передаёт подготовленный SQL через второй выход узлу записи файла, затем exec запускает sqlite3 с файлами схемы и данных.

6.2. Функция разбора журнала

Функция разбивает журнал на строки и извлекает поля. Номер n должен быть положительным целым числом не более 1 000 000 000. Допускаются типы EVENT THRESHOLD и NORMAL. При неверном номере или неизвестном типе события обработчик сообщает об ошибке и не выдаёт SQL.

Для DATA каждый s1, s2 и s3 проверяется отдельно. Конечное число записывается в readings даже при выходе за диапазон. Если значение отсутствует, не является конечным числом, выходит за диапазон или для s3 не равно 0 или 1, формируется событие FAULT. Для отсутствующего или нечислового значения запись readings не создаётся, но остальные значения строки могут быть загружены.

Если в журнале нет ни одной строки DATA, загрузка отменяется. В остальных допустимых случаях SQL очищает обе таблицы и сбрасывает счётчики, после чего в транзакции записывает новый снимок журнала. Повторная загрузка того же файла не накапливает дубликаты. Функция использует второй выход: return [null, msg].

6.3. Запуск

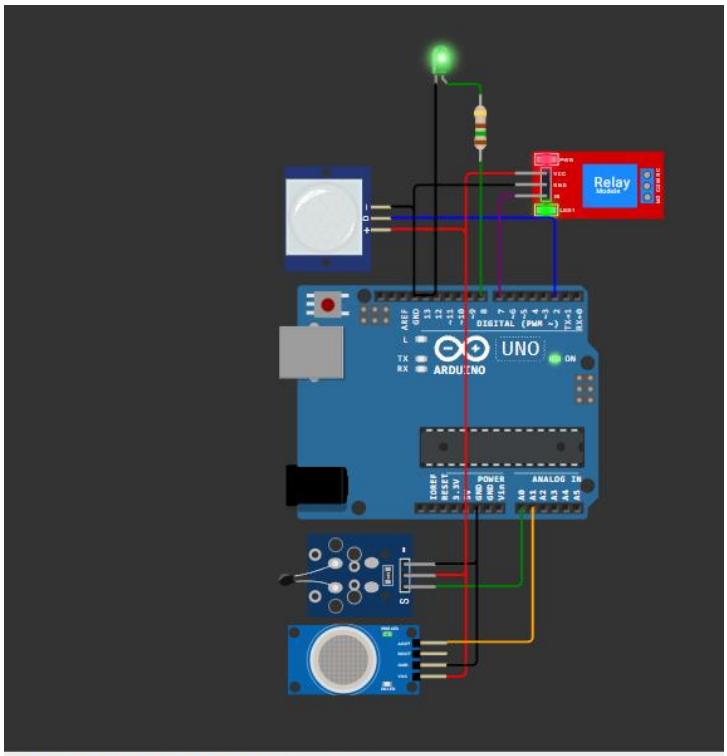
Из папки проекта в Windows запускается start-nodered.bat; затем открывается http://localhost:1880, импортируется nodered/flow.json и выполняется Deploy. Вывод Serial Monitor из Wokwi сохраняется в data/log.txt. Для контрольных запросов из корня проекта применяется команда:

```
sqlite3 -header -column db/variant.db ".read db/queries.sql"
```

Для ручной загрузки используются скрипты db/schema.sql и db/load.sql. Рабочая папка проекта определяет относительные пути к файлам.

7. Тестирование и результаты

Для проверки использованы предоставленные файлы проекта, скриншоты симуляции и выполненные отдельно программные проверки обработчика и SQL. Исходная база и проект при повторной проверке не изменялись.




```
DATA n=14 s1=30.91 s2=88.66 s3=0
EVENT n=14 type=THRESHOLD
DATA n=15 s1=30.91 s2=88.66 s3=0
DATA n=16 s1=30.91 s2=88.66 s3=0
DATA n=17 s1=30.91 s2=88.66 s3=0
DATA n=18 s1=30.91 s2=88.66 s3=0
DATA n=19 s1=30.91 s2=88.66 s3=0
```

Рисунок 5 — Работающая симуляция Wokwi

На снимке видны показания s1 = 30,91 °C, s2 = 88,66 %, s3 = 0 и событие THRESHOLD при n = 14. Зелёный индикатор включён.

7.1. Штатный сценарий

В предоставленном data/log.txt содержатся 11 строк DATA с номерами n = 3…13 и два события. При n = 6 температура возрастает до 42,46 °C; при n = 11 возвращается до 19,49 °C. В обработчике и тестовой SQLite-базе получены 33 показания и 2 события.

Таблица 9 — Фрагмент штатного журнала

n	t_s, c	s1, °C	s2, %	s3	Событие
3	9	23,99	88,66	0	—
4	12	23,99	88,66	0	—
5	15	23,99	88,66	0	—
6	18	42,46	88,66	0	THRESHOLD
7	21	42,46	88,66	0	—
8	24	42,46	88,66	0	—
9	27	42,46	88,66	0	—
10	30	42,46	88,66	0	—
11	33	19,49	88,66	0	NORMAL
12	36	19,49	88,66	0	—
13	39	19,49	88,66	0	—

Оба события согласуются с изменением температуры относительно порога 27 °C. Показания всех трёх датчиков сохранены в журнале.

Результаты контрольных запросов

Таблица 10 — Десять последних показаний

id	n	t_s	sensor	value
33	13	39	s3	0,0
32	13	39	s2	88,66
31	13	39	s1	19,49
30	12	36	s3	0,0
29	12	36	s2	88,66
28	12	36	s1	19,49
27	11	33	s3	0,0
26	11	33	s2	88,66
25	11	33	s1	19,49
24	10	30	s3	0,0

Таблица 11 — Минимум, среднее и максимум

Датчик	min_v	avg_v	max_v
s1	19,49	31,16	42,46
s2	88,66	88,66	88,66
s3	0,0	0,0	0,0

Таблица 12 — События штатного журнала

n	t_s, c	type
6	18	THRESHOLD
11	33	NORMAL

Средние значения вычисляются отдельно для каждого датчика. При включении в выборку аварийных числовых значений, таких как 999, обычный запрос AVG также их учитывает.

7.2. Сценарий неисправности датчика

Для проверки выхода за диапазон использован data/log_fault.txt:

```
DATA n=1 s1=999 s2=50.00 s3=0
```

Число 999 превышает допустимые 60 °C для s1. В отдельной базе после обработки получены три показания и одно событие: n = 1, t_s = 3, type = FAULT, message = «датчик s1 вне диапазона». Результат контрольного запроса пользователя показан на рисунке 6.

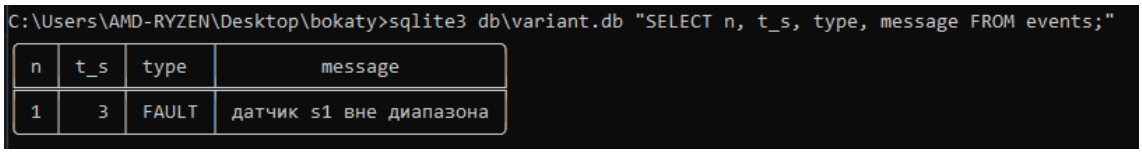


Рисунок 6 — Результат проверки FAULT

7.3. Повторная загрузка и обработка ошибок

При повторном выполнении SQL штатного журнала в тестовой базе остаются 33 строки readings и 2 события: база хранит снимок файла. Для строки с s1=nan обработчик сформировал две строки показаний для s2 и s3 и одно событие FAULT; пустой журнал был отклонён.

Таблица 13 — Сводка проверок

Проверка	Получено
Штатный журнал	33 показания; 2 события
Повторная загрузка	33 показания; 2 события
s1=999	3 показания; 1 FAULT
s1=nan	2 показания; 1 FAULT
Пустой журнал	Загрузка отменена

8. Расчёт объёма данных

Для предварительной оценки предполагается непрерывное накопление показаний с периодом 3 с. Один полный цикл создаёт три строки readings. Условный размер одной записи принят равным 40 Б; фактический размер зависит от устройства SQLite, страниц и индексов.

Таблица 14 — Расчёт объёма накопления

Показатель	Расчёт	Результат
Циклов в сутки	86 400 / 3	28 800
Показаний в сутки	28 800 × 3	86 400
За сутки	86 400 × 40 Б	≈ 3,456 МБ
За 30 суток	3,456 × 30	≈ 103,68 МБ
За 365 суток	3,456 × 365	≈ 1,261 ГБ

Использованы десятичные единицы: 1 МБ = 10^6 Б, 1 ГБ = 10^9 Б. События, индексы, внутренние структуры и размер исходного текстового журнала в оценку не включены. Число событий зависит от изменения температуры и ошибок.

Оценка за месяц и год приведена для накопительного режима. В текущем потоке содержимое таблиц представляет снимок загруженного журнала; размер базы определяется объёмом этого файла.

9. Управление версиями Git

Проект хранится в Git. Файлы разделены по каталогам firmware, sim, db, nodered, data и docs. История коммитов, приведённая ниже, получена из предоставленного архива проекта.

Таблица 15 — История коммитов предоставленного архива

Хеш	Дата	Сообщение
b9e92fa	2026-11-05	Структура проекта, README, .gitignore
957179e	2026-11-05	Добавлена структурная схема
5a7a742	2026-11-05	Схема и модель Wokwi
02e8c14	2026-11-05	Прошивка Arduino и алгоритм
df8e932	2026-11-05	Схема базы данных и запросы
00e898e	2026-11-05	Загрузка данных через Node-RED
16ffa75	2026-11-05	Журналы и проверка неисправности

Изменения удобно фиксировать отдельными завершёнными этапами. Перед коммитом следует проверить git status, состав индекса и исключения из .gitignore. Полный вывод истории приведён в приложении К. История отражает последовательность разработки проекта по состоянию на предоставленный архив.

10. Предложения по модернизации

Для повышения устойчивости управления целесообразно добавить гистерезис: включать выход при одном пороге температуры, а выключать при более низком. Это уменьшит частые переключения при колебаниях около 27 °C.

Контроль датчиков можно дополнить проверкой обрыва и короткого замыкания, временем актуальности показаний и отдельным сигналом отказа. Для газового датчика возможна дополнительная калибровка шкалы.

Для длительного наблюдения можно организовать автоматический приём последовательного потока, календарные временные метки, накопительное хранение и резервное копирование.

Выводы

В курсовом проекте рассмотрена виртуальная система сбора и учёта данных для серверной, вариант № 3. Определены подключения NTC, MQ2 и PIR к Arduino Uno и условие температурного события s1 > 27 °C. Описаны структурная схема, алгоритм и формат сообщений DATA/EVENT.

Выполнен расчёт светодиодной цепи: при 5 В, 2,0 В и 20 мА получен резистор 150 Ом. Расчётный ток системы составил 343,5 мА, а с запасом 30 % — около 446,6 мА. Оценены шаг АЦП и особенности нелинейного преобразования температуры.

Спроектированы таблицы readings и events и подготовлены контрольные запросы. Функция Node-RED проверяет данные и регистрирует неисправности FAULT; при загрузке формируется снимок журнала. Полный поток Node-RED подготовлен для импорта и содержит узлы чтения, проверки и загрузки.

Штатный журнал дал 33 показания и два события, тест s1 = 999 — три показания и одно событие FAULT. Снимок работающей Wokwi-модели показывает превышение порога и включённый индикатор. Условная оценка накопления показаний составляет около 3,456 МБ в сутки без служебных расходов.

Список используемых сокращений

Сокращение	Расшифровка
АЦП	Аналого-цифровой преобразователь
БД	База данных
ЕСКД	Единая система конструкторской документации
СУБД	Система управления базами данных
LED	Светодиод
MQ2	Газочувствительный датчик
NTC	Терморезистор с отрицательным температурным коэффициентом
PIR	Пассивный инфракрасный датчик движения
SQL	Язык структурированных запросов
USB	Универсальная последовательная шина
NaN	Нечисловое значение (Not a Number)

Список используемой литературы

1. Петин В.А. Arduino и Raspberry Pi в проектах Internet of Things. — СПб.: БХВ-Петербург, 2016.
2. Arduino. Language Reference; Arduino Uno Rev3. — URL: https://docs.arduino.cc/language-reference/ ; https://docs.arduino.cc/hardware/uno-rev3/.
3. Wokwi. Documentation. — URL: https://docs.wokwi.com/.
4. Node-RED. Documentation. — URL: https://nodered.org/docs/.
5. SQLite. Documentation; Command Line Shell. — URL: https://www.sqlite.org/docs.html ; https://www.sqlite.org/cli.html.
6. Чакон С., Штрауб Б. Pro Git. — 2-е изд. — URL: https://git-scm.com/book/ru/v2.
7. ГОСТ 2.701-2008. ЕСКД. Схемы. Виды и типы. Общие требования к выполнению.
8. EasyEDA. Documentation. — URL: https://docs.easyeda.com/.
9. Microchip Technology. ATmega328P. — URL: https://www.microchip.com/en-us/product/ATmega328P.
10. BISS0001. Техническое описание микросхемы обработки сигнала PIR (предоставленный документ DOC003842916.pdf).

Приложение А. Структурная схема

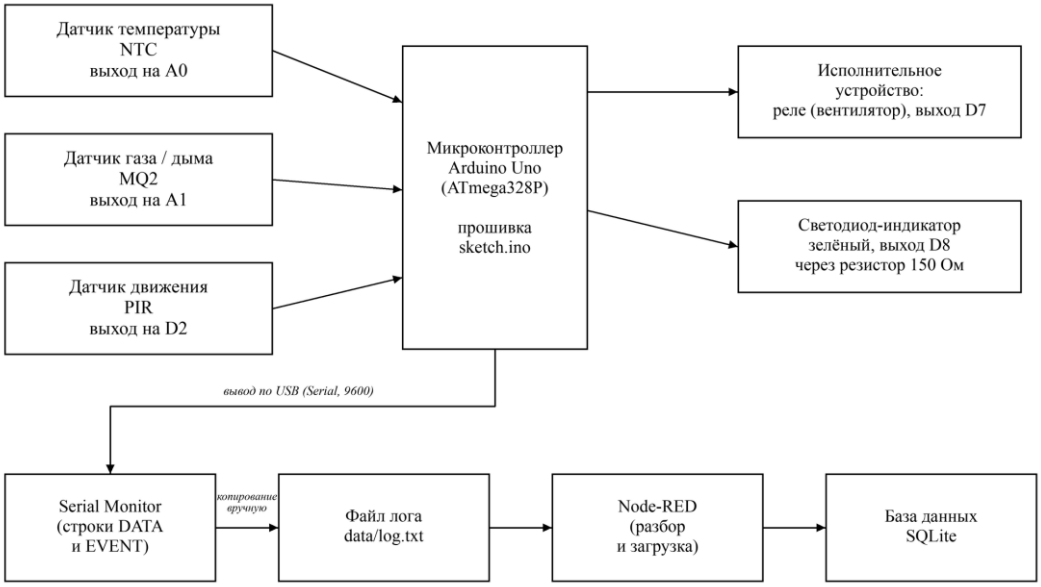
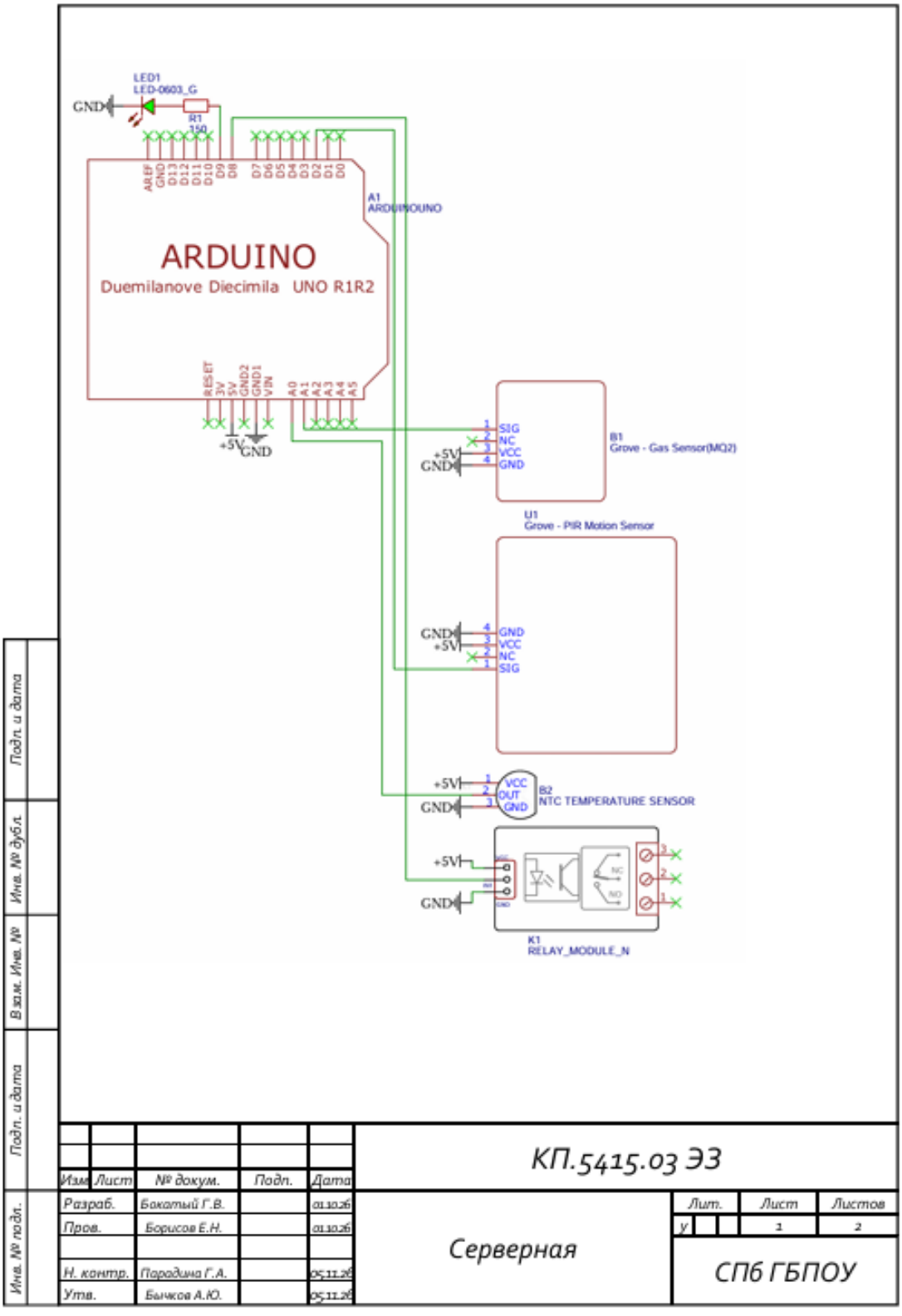


Рисунок А.1 — Структурная схема системы серверной

Обозначения выводов соответствуют прошивке firmware/sketch.ino. Перенос текста из Serial Monitor в data/log.txt выполняется вручную.

Приложение Б. Схема электрическая принципиальная



Приложение В. Перечень элементов

Форм.	Зона	Поз.	Обозначение	Наименование	Кол.	Примеч.
				Модули и платы		
			A1	Плата Arduino Uno	1	
			B1	Газовый датчик, MQ2	1	
			B2	Температурный датчик, NTC	1	
			K1	Реле, RELAY_MODULE	1	
			U1	Датчик движения, PIR		
			LED1	Светодиод LED5v, зелёный	1	
			R1	Резистор, 150	1	

Изм	Лист	№докум.	Подп.	Дата	КП.5415.03 ПЭ3		
Разраб.		Бокатый Г.В.		05.12.25	Серверная	Лит.	Лист 2 / Листов 2
Пров.		БарковЕ.Н.		05.12.25			
Н.контр.		Парадина Г.А.		05.12.25		СПб ГБПОУ	
Утв.		Бычков А.Ю.		05.12.25			

Приложение Г. Модель в Wokwi

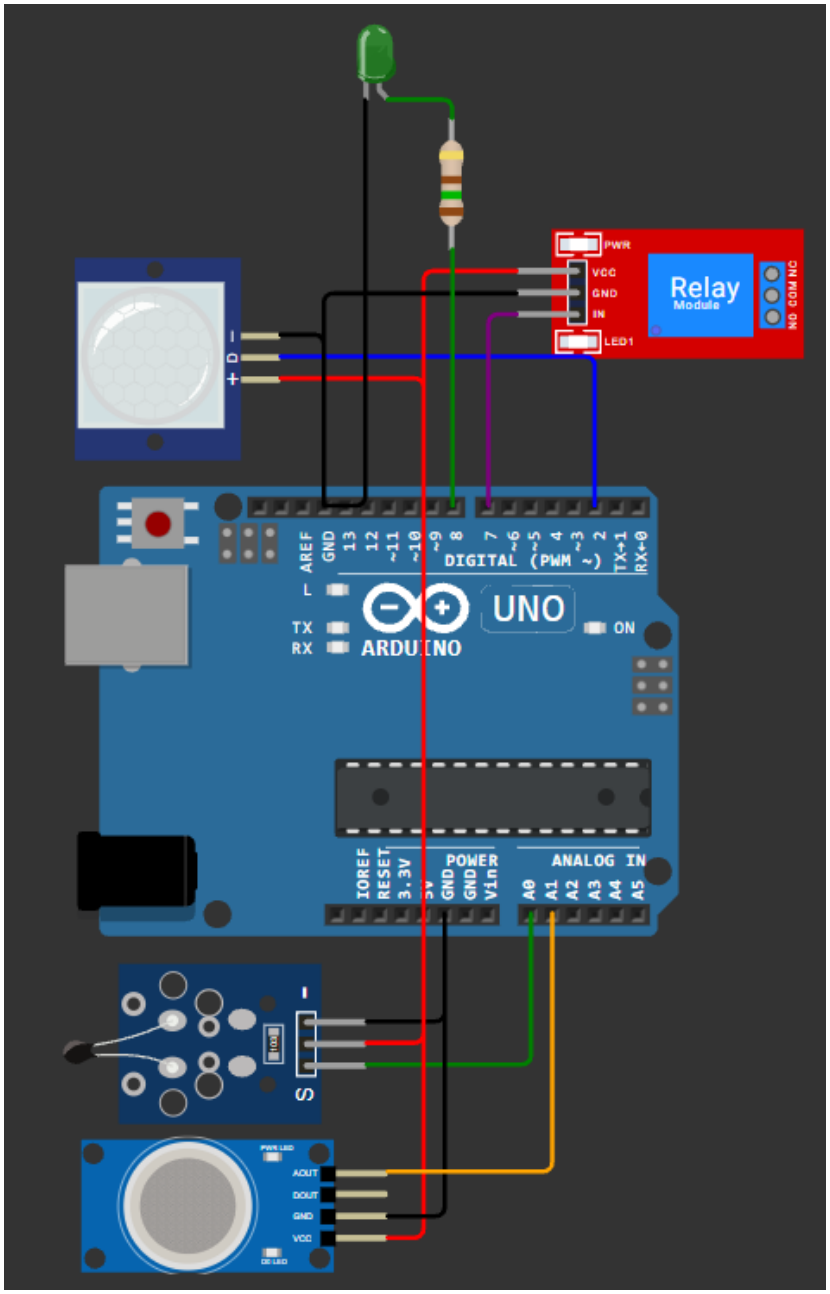


Рисунок Г.1 — Виртуальная модель Arduino Uno

Модель представлена файлом sim/diagram.json. Работа модели с выводом Serial Monitor показана в разделе 7.

Приложение Д. Листинг программы Arduino

```
// Вариант 3: Серверная
// Датчики: NTC (температура 0-60 C) -> A0, MQ2 (дым 0-100 %) -> A1, PIR (присутствие) -> D2
// Исполнительное устройство: реле (вентилятор) -> D7, светодиод зелёный (150 Ом) -> D8
// Период опроса T = 3 с. Порог события: температура > 27 C
const unsigned long T_MS = 3000;     // период опроса, мс
const float BETA = 3950.0;           // коэффициент Beta NTC в Wokwi
const float SMOKE_MIN = 0.0;         // диапазон датчика 2, %
const float SMOKE_MAX = 100.0;
const float TEMP_THRESHOLD = 27.0;   // порог события, C
const uint8_t PIN_NTC   = A0;
const uint8_t PIN_MQ2   = A1;
const uint8_t PIN_PIR   = 2;
const uint8_t PIN_RELAY = 7;
const uint8_t PIN_LED   = 8;
unsigned long lastMs = 0;
unsigned long n = 0;
bool thresholdActive = false;
// Температура NTC из analogRead (формула Beta из документации Wokwi).
// У NTC в Wokwi значение analogRead падает при росте температуры.
float readTemperature() {
  int a = analogRead(PIN_NTC);
  if (a < 1) a = 1;
  if (a > 1022) a = 1022;
  return 1.0 / (log(1.0 / (1023.0 / a - 1.0)) / BETA + 1.0 / 298.15) - 273.15;
}
void setup() {
  Serial.begin(9600);
  pinMode(PIN_PIR, INPUT);
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_RELAY, LOW);
  digitalWrite(PIN_LED, LOW);
}
void loop() {
  unsigned long now = millis();
  if (now - lastMs < T_MS) return;
  lastMs += T_MS;
  n++;
  float s1 = readTemperature();
  // дым: значение = мин + (макс - мин) * analogRead / 1023
  float s2 = SMOKE_MIN + (SMOKE_MAX - SMOKE_MIN) * analogRead(PIN_MQ2) / 1023.0;
  int   s3 = digitalRead(PIN_PIR);
  Serial.print("DATA n=");  Serial.print(n);
  Serial.print(" s1=");     Serial.print(s1, 2);
  Serial.print(" s2=");     Serial.print(s2, 2);
  Serial.print(" s3=");     Serial.println(s3);
  bool over = (s1 > TEMP_THRESHOLD);
  if (over && !thresholdActive) {
    thresholdActive = true;
    digitalWrite(PIN_RELAY, HIGH);
    digitalWrite(PIN_LED, HIGH);
    Serial.print("EVENT n="); Serial.print(n); Serial.println(" type=THRESHOLD");
  } else if (!over && thresholdActive) {
    thresholdActive = false;
    digitalWrite(PIN_RELAY, LOW);
    digitalWrite(PIN_LED, LOW);
    Serial.print("EVENT n="); Serial.print(n); Serial.println(" type=NORMAL");
  }
}
```

Приложение Е. Листинг функции Node-RED

```
// Вариант 3. Время условное: номер цикла × 3 секунды.
const T = 3;
const ranges = {s1:[0,60], s2:[0,100], s3:[0,1]};
const text = String(msg.payload ?? '').replace(/^\uFEFF/, '');
const lines = text.split(/\r?\n/);
const statements = [];
let count = 0;
const numberPattern = /^[+-]?(?:\d+(?:\.\d*)?|\.\d+)(?:[eE][+-]?\d+)?$/;
for (let index = 0; index < lines.length; index++) {
    const line = lines[index].trim();
    if (!line) continue;
    const parts = line.split(/\s+/);
    const kind = parts.shift();
    if (kind !== 'DATA' && kind !== 'EVENT') continue;
    const f = {};
    for (const part of parts) {
        const eq = part.indexOf('=');
        if (eq > 0) f[part.slice(0,eq)] = part.slice(eq+1);
    }
    if (!/^\d+$/.test(f.n || '') || !Number.isSafeInteger(Number(f.n)) || Number(f.n) < 1 ||
Number(f.n) > 1000000000) {
        node.error('Некорректный номер n в строке ' + (index + 1) + '. База не изменена.', msg);
        return null;
    }
    const n = Number(f.n), time = n * T;
    if (kind === 'DATA') {
        count++;
        for (const sensor of ['s1','s2','s3']) {
            const raw = f[sensor];
            const value = raw !== undefined && numberPattern.test(raw) ? Number(raw) : NaN;
            if (Number.isFinite(value)) {
```

Приложение Е. Листинг функции Node-RED (продолжение)

```
                statements.push(`INSERT INTO readings(n,t_s,sensor,value)
VALUES(${n},${time},'${sensor}',${value});`);
            }
            if (!Number.isFinite(value) || value < ranges[sensor][0] || value > ranges[sensor][1]
|| (sensor === 's3' && value !== 0 && value !== 1)) {
                statements.push(`INSERT INTO events(n,t_s,type,message)
VALUES(${n},${time},'FAULT','датчик ${sensor} вне диапазона');`);
            }
        }
    } else {
        if (!['THRESHOLD','NORMAL'].includes(f.type)) {
            node.error('Неизвестный тип EVENT в строке ' + (index + 1) + '. База не изменена.',
msg);
            return null;
        }
        statements.push(`INSERT INTO events(n,t_s,type,message)
VALUES(${n},${time},'${f.type}','');`);
    }
}
if (!count) {
    node.warn('В журнале нет строк DATA. Загрузка отменена: база не изменена.');
    return null;
}
const head = "BEGIN;\nDELETE FROM readings;\nDELETE FROM events;\nDELETE FROM sqlite_sequence
WHERE name IN ('readings','events');\n";
msg.payload = head + statements.join('\n') + '\nCOMMIT;\n';
node.status({fill:'green',shape:'dot',text:'Строк DATA: ' + count});
return [null, msg];
```

Приложение Ж. Скрипты базы данных

db/schema.sql

```
CREATE TABLE IF NOT EXISTS readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    n INTEGER NOT NULL,
    t_s INTEGER NOT NULL,
    sensor TEXT NOT NULL,
    value REAL NOT NULL
);

CREATE TABLE IF NOT EXISTS events (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    n INTEGER NOT NULL,
    t_s INTEGER NOT NULL,
    type TEXT NOT NULL,
    message TEXT
);
```

db/queries.sql

```
-- 1. Десять последних записей показаний
SELECT * FROM readings ORDER BY id DESC LIMIT 10;

-- 2. Минимум, среднее и максимум для каждого датчика
SELECT sensor,
       MIN(value) AS min_v,
       ROUND(AVG(value), 2) AS avg_v,
       MAX(value) AS max_v
FROM readings
GROUP BY sensor;

-- 3. Все события в порядке добавления
SELECT * FROM events ORDER BY id;
```

Приложение И. Журналы и загрузочный скрипт

```
data/log.txt — штатный журнал
DATA n=3 s1=23.99 s2=88.66 s3=0
DATA n=4 s1=23.99 s2=88.66 s3=0
DATA n=5 s1=23.99 s2=88.66 s3=0
DATA n=6 s1=42.46 s2=88.66 s3=0
EVENT n=6 type=THRESHOLD
DATA n=7 s1=42.46 s2=88.66 s3=0
DATA n=8 s1=42.46 s2=88.66 s3=0
DATA n=9 s1=42.46 s2=88.66 s3=0
DATA n=10 s1=42.46 s2=88.66 s3=0
DATA n=11 s1=19.49 s2=88.66 s3=0
EVENT n=11 type=NORMAL
DATA n=12 s1=19.49 s2=88.66 s3=0
DATA n=13 s1=19.49 s2=88.66 s3=0

data/log_fault.txt — тест неисправности
DATA n=1 s1=999 s2=50.00 s3=0


db/load.sql — содержимое предоставленного архива
BEGIN;
DELETE FROM readings;
DELETE FROM events;
DELETE FROM sqlite_sequence WHERE name IN ('readings','events');
INSERT INTO readings(n,t_s,sensor,value) VALUES(3,9,'s1',23.99);
INSERT INTO readings(n,t_s,sensor,value) VALUES(3,9,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(3,9,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(4,12,'s1',23.99);
INSERT INTO readings(n,t_s,sensor,value) VALUES(4,12,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(4,12,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(5,15,'s1',23.99);
INSERT INTO readings(n,t_s,sensor,value) VALUES(5,15,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(5,15,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(6,18,'s1',42.46);
INSERT INTO readings(n,t_s,sensor,value) VALUES(6,18,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(6,18,'s3',0);
```

Приложение И. Журналы и загрузочный скрипт (продолжение)

```
INSERT INTO events(n,t_s,type,message) VALUES(6,18,'THRESHOLD','');
INSERT INTO readings(n,t_s,sensor,value) VALUES(7,21,'s1',42.46);
INSERT INTO readings(n,t_s,sensor,value) VALUES(7,21,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(7,21,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(8,24,'s1',42.46);
INSERT INTO readings(n,t_s,sensor,value) VALUES(8,24,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(8,24,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(9,27,'s1',42.46);
INSERT INTO readings(n,t_s,sensor,value) VALUES(9,27,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(9,27,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(10,30,'s1',42.46);
INSERT INTO readings(n,t_s,sensor,value) VALUES(10,30,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(10,30,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(11,33,'s1',19.49);
INSERT INTO readings(n,t_s,sensor,value) VALUES(11,33,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(11,33,'s3',0);
INSERT INTO events(n,t_s,type,message) VALUES(11,33,'NORMAL','');
INSERT INTO readings(n,t_s,sensor,value) VALUES(12,36,'s1',19.49);
INSERT INTO readings(n,t_s,sensor,value) VALUES(12,36,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(12,36,'s3',0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(13,39,'s1',19.49);
INSERT INTO readings(n,t_s,sensor,value) VALUES(13,39,'s2',88.66);
INSERT INTO readings(n,t_s,sensor,value) VALUES(13,39,'s3',0);
COMMIT;
```

Приложение К. История Git

Фактический вывод из предоставленного архива, от последнего коммита к первому:

```
16ffa75  2026-11-05  Журналы и проверка неисправности
00e898e  2026-11-05  Загрузка данных через Node-RED
df8e932  2026-11-05  Схема базы данных и запросы
02e8c14  2026-11-05  Прошивка Arduino и алгоритм
5a7a742  2026-11-05  Схема и модель Wokwi
957179e  2026-11-05  Добавлена структурная схема
b9e92fa  2026-11-05  Структура проекта, README, .gitignore
```

История отражает состояние предоставленного архива проекта.

Командный файл start-nodered.bat:

```
@echo off
rem Starts Node-RED from the project folder.
set TOOLS=C:\Users\AMD-RYZEN\Desktop\tools
set PATH=%TOOLS%\node;%TOOLS%\sqlite;%PATH%
cd /d %~dp0
node %TOOLS%\nodered\node_modules\node-red\red.js --userDir %TOOLS%\nodered-data
pause
```